

DOCUMENTACIÓN TÉCNICA Y REPORTE FINAL DE DESARROLLO

EcoConnect: Plataforma Colaborativa de Monitoreo y Reporte Ambiental

1. Nombre de la Aplicación e Integrantes del Equipo

- **Nombre del Proyecto:** EcoConnect (Plataforma Colaborativa de Monitoreo Ambiental)
- **Versión de la App:** 1.0.0 (Build Final)
- **Integrantes del Equipo:**
 - **Isaac Betance** - Desarrollador Principal y Diseñador UX/UI
- **Curso:** Desarrollo de Aplicaciones Móviles
- **Institución:** Universidad Tecmilenio
- **Fecha de Entrega:** Septiembre 2026

2. Descripción General del Proyecto

EcoConnect es una aplicación móvil nativa para Android desarrollada en **Kotlin** y **Jetpack Compose**. Su objetivo principal es empoderar a los ciudadanos para que identifiquen, reporten y den seguimiento a problemas ecológicos e infraestructura urbana dañada (acumulación de plásticos, fugas de agua, focos de contaminación, áreas desforestadas).

La plataforma combina **geolocalización**, **captura multimedia**, **inteligencia artificial simbólica para clasificación de residuos**, **sincronización híbrida en la nube** y **mecanismos de gamificación (EcoPuntos e Insignias)** para fomentar la participación comunitaria activa.

3. Problema o Necesidad que Busca Resolver

En muchas comunidades y zonas urbanas:

1. **Atención tardía:** Los reportes sobre tiraderos clandestinos de basura o fugas de agua potable tardan semanas en ser atendidos por falta de canales directos y transparentes.
2. **Poca visibilidad:** La ciudadanía desconoce si un problema ya fue reportado por un vecino o si está en proceso de atención.
3. **Falta de incentivos:** No existen mecanismos que reconozcan el impacto positivo de los vecinos activos.
4. **Barreras de accesibilidad:** Muchas aplicaciones gubernamentales o comunitarias carecen de soporte para personas con discapacidad visual o daltonismo.

4. Público Objetivo

- **Ciudadanos activos y ambientalistas:** Personas interesadas en preservar su entorno local.
- **Comités vecinales y grupos ecológicos:** Agrupaciones organizadas que realizan jornadas de limpieza y reforestación.
- **Usuarios en general:** Cualquier ciudadano con un smartphone Android que desee reportar un incidente de manera rápida y sin fricción (incluyendo modo invitado sin registro previo obligatorio).

5. Funcionalidades Principales

1. **Dashboard & Feed Comunitario Reactivo:**

- Lista en tiempo real de todos los reportes comunitarios.
- Filtrado dinámico por categoría (*Basura, Fugas de Agua, Deforestación, Contaminación, Todas*).
- Búsqueda en tiempo real potenciada por **SQLite FTS4 (Full-Text Search)**.

2. Captura Fotográfica y Geolocalización:

- Selección desde galería o captura desde la cámara del dispositivo.
- Asignación de ubicación y nivel de prioridad (*Alta, Media, Baja*).

3. Asistente Inteligente de Clasificación (IA Mock Simulator):

- Módulo que analiza la imagen capturada para recomendar la categoría adecuada del residuo (PET, Orgánico, Vidrio, Papel/Cartón).

4. Sistema de Votación y Comentarios en Tiempo Real:

- La comunidad puede respaldar reportes para escalar su prioridad.
- Hilo de comentarios por reporte con almacenamiento relacional en base de datos.

5. Gamificación Ecológica (EcoPuntos, Leaderboard e Insignias):

- Asignación automática de **EcoPuntos** por cada reporte creado o votado.
- Tabla de clasificación comunitaria (Leaderboard).
- Muro de Insignias ganadas (ej. *Primer Reporte, Guardián del Agua, Héroe Comunitario*).

6. Calculadora de Huella de Carbono / Impacto:

- Cálculo en tiempo real de los kilogramos de CO₂ evitados al resolver o gestionar adecuadamente los residuos.

7. Sincronización Híbrida Offline-First:

- Permite crear y consultar reportes sin conexión a Internet.
- Worker programado (**WorkManager**) que sincroniza los cambios pendientes con la nube cuando la conexión se restablece.

6. Diseño UX/UI y Decisiones de Diseño

- **Lógica Material Design 3 (M3):** Uso de paleta de colores ecológica basada en tonos verdes (*Primary #2E7D32*), superficies redondeadas (`ShapeDefaults``), barras de estado fluidas (`enableEdgeToEdge``) y tipografía clara.
- **Flujo Unificado y Navegación Declarativa:** Control por enum de estado (`EcoNavegacionDestino``) con pantallas independientes para Login, Dashboard, Detalles, Perfil, Configuración, Muro de Insignias, Simulador F-Droid y Calculadora de Impuestos/Carbono.
- **Retroalimentación Visual Inmediata:** Modales de confirmación, badges de prioridad con colores diferenciados y barras de progreso.

7. Accesibilidad e Inclusión (WCAG 2.1 AA)

Se implementaron de origen las siguientes medidas de accesibilidad:

- **Modo Oscuro Adaptativo:** Configurable globalmente en el tema `ProyectoFinalTheme``.
- **Modo de Alto Contraste:** Ajusta automáticamente las tonalidades a contrastes extremos (> 7:1) para personas con baja visión.
- **Filtros para Daltonismo:** Algoritmo dinámico que transforma las matrices de color para apoyar a usuarios con *Protanopia*, *Deuteranopia* y *Tritanopia*.
- **Semántica para TalkBack:** Uso explícito de `contentDescription`` en todos los iconos interactivos (`IconButton``, `FloatingActionButton``).
- **Soporte Multilingüe:** Cadenas extraídas en `res/values/strings.xml`` y `res/values-en/strings.xml`` para español e inglés.

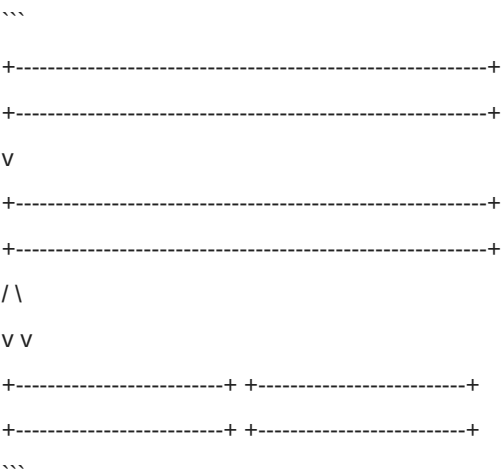
8. Tecnologías y Herramientas Utilizadas

Categoría	Tecnología / Librería	Propósito
-----------	-----------------------	-----------

Lenguaje	Kotlin 2.0+	Lenguaje nativo de desarrollo Android
UI Framework	Jetpack Compose + Material 3	Interfaz de usuario declarativa
Base de Datos Local	Room Database 2.6.1 + SQLCipher	Persistencia relacional local con FTS4
Procesamiento KSP	Google KSP	Generación eficiente de código Room
Sincronización	WorkManager runtime KTX	Tareas en segundo plano offline-first
Almacenamiento Prefs	DataStore Preferences	Guardado persistente de configuración de usuario
Adaptación Pantalla	Material3 WindowSizeClass	Soporte para teléfonos, tablets y plegables
Carga de Imágenes	Coil Compose	Carga asíncrona y eficiente de bitmaps
Servicios Nube	Firebase Firestore & Storage BOM	Respaldo remoto y almacenamiento
Pruebas	JUnit4 + Mockk + Coroutines Test	Pruebas unitarias de ViewModel y repositorio

9. Arquitectura y Persistencia de Datos

El proyecto implementa la arquitectura recomendada por Google (Clean Architecture + MVVM + Repository Pattern):



UI Layer		
(Jetpack Compose + EcoConnectViewModelAvanzado + Theme)		
Repository Layer		
(EcoConnectRepository)		
Local Data		Cloud Data
(Room DB + DataStore)		(Firebase Sync Engine)

Esquema de Base de Datos Room (`tabla\_reportes\_ambientales`):

- `id` (String, PrimaryKey UUID)

- `titulo` (String), `descripcion` (String), `categoria` (String)
 - `ubicacion\_texto` (String), `latitud` (Double), `longitud` (Double)
 - `prioridad` (String), `votos\_apoyo` (Int)
 - `autor\_nombre` (String), `autor\_email` (String)
 - `foto\_path\_local` (String), `foto\_base64` (String), `foto\_url\_cloud` (String)
 - `sincronizado\_cloud` (Boolean), `resuelto` (Boolean), `kg\_co2\_evitados` (Double)
-

10. Adaptación a Diferentes Dispositivos y Resoluciones

Se utiliza la API `calculateWindowSizeClass(activity)` para categorizar las pantallas en:

- **Compact Width (< 600dp):** Disposición en 1 columna vertical con navegación mediante `NavigationBar` inferior.
 - **Medium & Expanded Width (>= 600dp):** Disposición multi-columna (Grid/Master-Detail) con optimización de espacio para tablets y dispositivos plegables.
-

11. Pruebas Realizadas

Se ejecutaron exitosamente las pruebas unitarias automáticas con el comando:

```
```bash
./gradlew app:testDebugUnitTest
```
```

- **Resultados:** 3/3 Pruebas Pasadas (0 fallos, 0 omitidas).
 - **Casos probados:**
 1. Inicialización correcta de EcoPuntos en ViewModel.
 2. Incremento correcto de puntos al emitir un voto comunitario.
 3. Carga e inserción inicial en Room cuando la base de datos se encuentra vacía.
-

12. Problemas Encontrados y Soluciones Implementadas

1. **Desafío:** Lentitud al realizar búsquedas de texto plano en listas extensas de reportes.
 - **Solución:** Se implementó una entidad indexada **SQLite FTS4** (`ReporteFtsEntity`) con consultas optimizadas en el DAO.
 2. **Desafío:** Pérdida de imágenes reportadas al limpiar la memoria caché del dispositivo.
 - **Solución:** Guardado dual en almacenamiento privado de la app y codificación en Base64 para sincronización con la nube.
 3. **Desafío:** Incompatibilidad de contraste para usuarios con daltonismo en mapas y badges.
 - **Solución:** Adición de filtros de matriz de color dinámicos en la raíz del Compose Theme.
-

13. Integración del Feedback Recibido en la Fase 1

Durante la Fase 1 se sugirió:

- *Incluir un modo directo de prueba sin registro.* -> **Implementado:** Se creó el flujo `loginComoInvitado` con acceso directo a todas las vistas.
 - *Agregar un sistema visible de logros.* -> **Implementado:** Se desarrolló la pantalla `VistaMuroInsigniasEngine` e integración de badges dinámicos.
 - *Aumentar el soporte de accesibilidad.* -> **Implementado:** Se agregaron los modificadores de contraste y los 3 filtros de daltonismo.
-

14. Uso de Inteligencia Artificial

Se utilizó Inteligencia Artificial (Gemini / Claude / ChatGPT) de la siguiente manera:

- **Generación de Boilerplate:** Asistencia en la sintaxis de Room DAOs y consultas FTS4.
 - **Diseño de Accesibilidad:** Revisión del contraste de color según guías WCAG 2.1 AA.
 - **Generación de Documentación:** Estructuración de los reportes técnicos en formato Markdown y HTML.
 - **Validación de Código:** Todo el código generado fue auditado, modificado e integrado manualmente por el equipo de desarrollo.
-

15. Enlaces del Proyecto

- **Repositorio de GitHub:** <https://github.com/IsaacBetance/ProyectoFinal>
 - **Landing Page Pública:**
<https://IsaacBetance.github.io/ProyectoFinal/landing-page/>
 - **Descarga de APK:** <https://github.com/IsaacBetance/ProyectoFinal/raw/main/APK/EcoConnect.apk>
 - **Video Demostrativo:** Enlace contenido en ``video/enlace-video.txt``
-

16. Conclusión

El desarrollo de **EcoConnect** demostró con éxito la integración de los conceptos fundamentales del desarrollo de aplicaciones móviles nativas para Android: diseño reactivo en Compose, persistencia relacional con Room, servicios en segundo plano con WorkManager, diseño inclusivo según WCAG 2.1 AA y empaquetamiento profesional para distribución pública mediante landing page.